

A Class of Efficient Contention Resolution Algorithms for Multiple Access Channels

JEANNINE MOSELY, MEMBER, IEEE, AND PIERRE A. HUMBLET, MEMBER, IEEE

Abstract—A discrete time multiaccess channel is considered where the outcome of a transmission is either “idle,” “success,” or “collision,” depending on the number of users transmitting simultaneously. Messages involved in a “collision” must be retransmitted. An efficient access allocation policy is developed for the case where infinitely many sources generate traffic in a Poisson manner and can all observe the outcomes of the previous transmissions. Its rate of success is 0.48776. Modifications are presented for the cases where the transmission times depend on the transmission outcomes and where observations are noisy.

I. INTRODUCTION

WE consider the following model of a multiple access channel. A large number of sources generate messages in a Poisson manner, at a total rate of λ messages per unit of time, starting at time 0. Once a message has been generated its source can transmit it on a common channel. Transmissions start at integer multiples of the unit of time and last one unit of time, also called a “slot.” If the transmissions from two or more sources overlap, a collision is said to occur, and all messages are lost and must be retransmitted at a later time. If only one source transmits, the transmission is successful.

All sources can observe the channel and learn at the end of a slot whether it is idle, or if a success or a collision has occurred. This common feedback is the only information the sources share. The problem is to find an efficient way of using the feedback to schedule the message transmissions and retransmissions.

The previous model is an idealization of practical communication systems [1]–[3] that have been the subject of numerous papers in the communication theory literature [4], [5]. Similar problems have also been treated in control theory journals [6]–[8]; indeed, such systems are nice examples of distributed control. Algorithms similar to the ones presented here have also been derived independently by Tsybakov and Mikhailov [9]. Reference [20] contains a good account of a variety of related algorithms along with numerous references.

In Section II, we present the basic algorithm and some of its properties. In Section III, we show how to maximize the throughput, i.e., the maximum long-term rate of success. In Section IV we discuss the implementation of the algorithm in real time. We then show, in Section V, how to modify and analyze the algorithm if the transmission times depend on the transmission outcomes. In Section VI we introduce the general first-generated first-transmitted algorithm. Finally, we treat the case where the feedback is noisy.

Paper approved by the Editor for Computer Communication of the IEEE Communications Society for publication after presentation in part at the 19th IEEE Conference on Decision and Control, Albuquerque, NM, 1980. Manuscript received April 14, 1982; revised August 15, 1983. This work was supported in part by the Defense Advanced Research Projects Agency under Contract ARPA ONR/N00014-75-1183 and by the National Science Foundation under Grant NSF ECS 79-19880. This paper is based in part on a thesis submitted by J. Mosely in partial fulfillment of the S.M. degree in electrical engineering from the Massachusetts Institute of Technology, Cambridge, June 1979.

The authors are with the Laboratory for Information and Decision Systems, Massachusetts Institute of Technology, Cambridge, MA 02139.

II. THE BASIC ALGORITHM

The algorithm defined below allows the transmission of the messages on the basis of their generation times. It has the advantage of being effective regardless of the number of sources, even infinite. It is a generalization of the procedure presented in [10], which was itself based on an idea from Hayes [11] and Capetanakis [12] that had been used previously for other applications as described in [13].

We can state the problem to be solved as follows: messages are generated according to a Poisson point process with rate λ on $R^+ = [0, \infty)$. At each step, the algorithm designates a subset of the half-line, and messages generated in that subset are transmitted. This subset is chosen as a function of the outcomes of all previous transmissions. The process is repeated *ad infinitum*. The rate at which successes are produced is called the “throughput” of the algorithm.

In our algorithm, the set of messages that are transmitted during the n th time unit interval are those generated in a time interval of the form $[a, b)$. This interval will be referred to as the “enabled interval.” At step n of the algorithm, we update three parameters, y_n , s_n , and t_n , which characterize the state of the algorithm. These parameters are used to calculate a and b , the endpoints of the enabled interval. Specifically, the enabled interval is given by $[a, b) = [y_n, y_n + F(s_n, t_n))$, where F is a function to be optimized below. F maps $R^+ \cup \{\infty\} \times R^+ \cup \{\infty\}$ into R^+ and is constrained to satisfy $F(s, t) < t$.

The values of y_0 , s_0 , and t_0 are initially equal to 0, ∞ , and ∞ , respectively. The y_n 's, s_n 's, and t_n 's ($s_n \leq t_n$) are updated by the following rule, where $F_n = F(s_n, t_n)$.

If a transmission results in

idle:

$$y_{n+1} \leftarrow y_n + F_n$$

$$s_{n+1} \leftarrow s_n - F_n$$

$$t_{n+1} \leftarrow t_n - F_n$$

success:

$$y_{n+1} \leftarrow y_n + F_n$$

$$s_{n+1} \leftarrow t_n - F_n$$

$$t_{n+1} \leftarrow \infty$$

collision:

$$y_{n+1} \leftarrow y_n$$

$$s_{n+1} \leftarrow \min(s_n, F_n)$$

$$t_{n+1} \leftarrow F_n.$$

As an example of how this algorithm works, consider Fig. 1. Here we choose F so that $F(\infty, \infty) = \tau$, where τ is some finite constant, $F(s, \infty) = s$, and $F(s, s) = s/2$ when s is finite. With the initial conditions for y , s , and t given above, it is easy to verify that for this F , the pair of parameters (s, t)

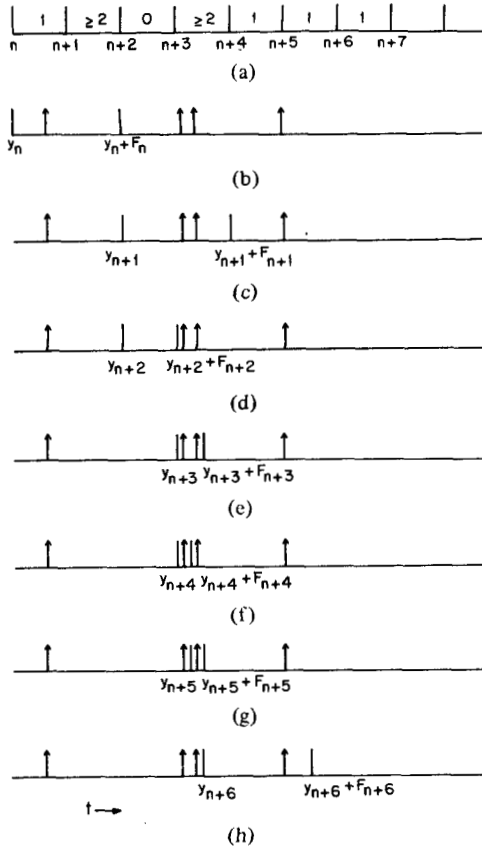


Fig. 1. Example of conflict resolution.

will always be one of (∞, ∞) , (s, ∞) , or (s, s) . This is Gallager's binary splitting algorithm [10].

In Fig. 1(a), the time line is divided into unit intervals and observation of the process begins at a time n , such that $s_n = t_n = \infty$. The number written above a slot indicates the transmission outcome for that slot: 0 represents an idle, 1 a successful transmission, and ≥ 2 a collision. Fig. 1(b)–(h) shows the sample process of message generation times that gives rise to the transmission outcome sequence of Fig. 1(a), together with the sequence of enabled intervals selected by the algorithm. In the n th slot there is a success, and the algorithm moves the enabled interval forward by $F_n = \tau$, as shown. In the $(n+1)$ st slot there is a collision, and so the enabled interval is split in half and the first half is tested for messages. Because there is an idle in slot $n+2$, this implies that the colliding messages were all generated in the second half of the $(n+1)$ st enabled interval. Hence, in slot $n+3$, it is desirable to examine only the third quarter of that interval. Since another collision occurs, this interval is split in two and the first half tested, yielding a success in slot $n+4$. Now it is known that the second half of the enabled interval for slot $n+3$ contains at least one message. Using it as the enabled interval for slot $n+5$ produces another success. At this time it can be observed that all messages generated between y_n and y_{n+6} have been successfully transmitted and all conflicts have been resolved. For slot $n+6$, the algorithm again selects the enabled interval of length τ as shown.

The algorithm used in this example is a special case of the general algorithm described above. Its choice of F will be seen to be slightly suboptimal with respect to throughput. It is not always desirable to divide an enabled interval containing a collision exactly in half. Also, when an interval is known to contain at least one message generation time, there exist conditions such that that interval is not the best choice

for the next enabled interval. These issues are discussed in Section III.

Returning to our general algorithm, we make two assertions about the message generation times. Given the outcomes of the n past transmissions, we know, for all n , that all messages generated in $[0, y_n)$ have been successfully transmitted. To see this, note that the monotonic sequence of times $y_0, y_1, \dots, y_n$ partitions the interval $[0, y_n)$ into the sequence of intervals $[y_0, y_1)$, $[y_1, y_2)$, $\dots$, $[y_{n-1}, y_n)$, such that each of these intervals contains at most one message generation time. (Whenever there is a collision, $y_{n+1} = y_n$ and the degenerate interval $[y_n, y_{n+1})$ is empty.) Hence, at time n , all messages generated prior to y_n must have been successfully transmitted.

It can also be shown that, given the outcomes of the n past transmissions, the message generation processes in $[0, y_n)$, $[y_n, y_n + t_n)$, and $[y_n + t_n, \infty)$ are independent. The generation times in $[y_n, y_n + t_n)$ are distributed according to a Poisson process with rate λ , conditioned on the facts that there is at least one generation time in $[y_n, y_n + s_n)$, and at least two generation times in $[y_n, y_n + t_n)$. The generation times in $[y_n + t_n, \infty)$ are distributed according to a Poisson process with rate λ .

In order to make a rigorous statement of these assertions, a few definitions are needed. Define: $A_n = [0, y_n)$, $B_n = [y_n, y_n + s_n)$, $C_n = [y_n, y_n + t_n)$, $D_n = [y_n + t_n, \infty)$, and $T_n = [y_n, y_n + F_n)$. Let $N(S)$ be the number of generation times in a set S . Let

$$\theta_n = \begin{cases} 0 & \text{if } N(T_n) = 0 \\ 1 & \text{if } N(T_n) = 1 \\ 2 & \text{if } N(T_n) \geq 2. \end{cases}$$

Let $\theta(n) = (\theta_1, \dots, \theta_n)$ and for convenience define $\theta(0)$ and $\theta(-1)$ arbitrarily so that we may condition on the events $\theta(0)$ and $\theta(-1)$. If we have an m -vector of sets $S = (S_1, \dots, S_m)$, for any set A , let $A \cap S$ denote the m -vector $(A \cap S_1, \dots, A \cap S_m)$ and let $N(S)$ denote $(N(S_1), \dots, N(S_m))$. We may now state the following.

Theorem: For any integers N_A, N_C, N_D , choose measurable finite subsets $A_{n_i} \subset A_n$ for $i = 1, \dots, N_A$; $C_{n_j} \subset C_n$ for $j = 1, \dots, N_C$; $D_{n_k} \subset D_n$ for $k = 1, \dots, N_D$. Then for any vectors $m \in \mathbb{Z}^{N_A}$, $p \in \mathbb{Z}^{N_C}$, $q \in \mathbb{Z}^{N_D}$,

$$\begin{aligned} \Pr(N(A_n) = m, N(C_n) = p, N(D_n) = q | \theta(n-1)) \\ = \Pr(N(A_n) = m | \theta(n-1)) \\ \cdot \Pr(N(C_n) = p | N(B_n) \geq 1, N(C_n) \geq 2) \Pr(N(D_n) = q) \quad (1) \end{aligned}$$

for all $n = 0, 1, \dots$. The probabilities in (1) can be calculated from the fact that the message arrival process is Poisson.

The proof of this theorem can be found in [14].

We offer here an intuitive justification for the theorem. If we assume that the theorem is true at some time n , we can show that the theorem is true for $n+1$ by considering separately each possible outcome for the transmission of the enabled interval T_n . For example, assume that the generation times in C_n are Poisson conditioned on C_n containing a conflict, and the generation times in D_n are known to be Poisson and are independent of those in C_n . Then, if the enabled interval T_n is found to contain a conflict, it is easy to show that the generation times in $C_{n+1} = T_n$ are Poisson conditioned on that interval containing a conflict. Surprisingly, the generation times in $D_{n+1} = [y_n + F_n, \infty)$ are also Poisson. To see this, note that $[y_n + F_n, \infty) = [y_n + F_n, y_n + t_n) \cup [y_n + t_n, \infty) = [y_n + F_n, y_n + t_n) \cup D_n$. The generation times in D_n were assumed to be Poisson, and the generation times in $[y_n + F_n, y_n + t_n)$ can be shown to be Poisson by the following informal argument. We use " k in T " as an abbrevia-

tion for "the event that there are k generation times in the interval I " and " ≥ 2 in I " as an abbreviation for "the event that there are at least two generation times in I ." Then,

$$\begin{aligned} & \Pr(k \text{ in } [y_n + F_n, y_n + t_n] | \geq 2 \text{ in } [y_n, y_n + F_n], \\ & \quad \geq 2 \text{ in } [y_n, y_n + t_n]) \\ &= \Pr(k \text{ in } [y_n + F_n, y_n + t_n] | \geq 2 \text{ in } [y_n, y_n + F_n], \\ & \quad \geq 0 \text{ in } [y_n + F_n, y_n + t_n]) \\ &= \Pr(k \text{ in } [y_n + F_n, y_n + t_n] | \geq 2 \text{ in } [y_n, y_n + F_n]) \\ &= \Pr(k \text{ in } [y_n + F_n, y_n + t_n]). \end{aligned}$$

Hence, D_{n+1} is the union of two disjoint intervals, each of which contains message generation times distributed according to independent Poisson processes.

In the next section we will show how to define $F(\cdot, \cdot)$ so as to maximize the rate of successful transmission.

III. ANALYSIS AND OPTIMIZATION

The key to the analysis of the algorithm is to realize that the process (s_n, t_n) is Markovian, as the probabilities of the different outcomes of the $(n+1)$ st transmission and the values of (s_{n+1}, t_{n+1}) depend only on s_n and t_n . This is a direct consequence of the theorem stated above, since the enabled interval T_n is a subset of C_n .

We should notice the peculiar role of the $(s_n, t_n) = (\infty, \infty)$ state. Physically it corresponds to all messages generated before y_n having been successfully transmitted, and no information except the *a priori* statistics being available about generation times greater than y_n . This state is entered every time two transmissions result in a success without an intervening conflict. Thus, it is reachable from all other states.

Moreover, if $F(\cdot, \cdot)$ is such that the probability of successful transmission in any state (s, t) has a positive lower bound (this is always the case for the $F(\cdot, \cdot)$'s considered below), then state (∞, ∞) is positive recurrent along with only countably many other states accessible from it. Thus, the computation of stationary state probabilities and expected values, with a given degree of precision, is a straightforward numerical matter.

We will now direct our attention to the problem of selecting $F(\cdot, \cdot)$ to maximize the long-term rate of success or throughput. That is, we wish to maximize

$$\liminf_{N \rightarrow \infty} \frac{1}{N} E \left(\sum_{n=1}^N I(\theta_n = 1) \right)$$

where I is the indicator function. (The indicator function is defined to be 1 if the indicated event occurs, and 0 otherwise.)

Throughout the analysis that follows, the parameters s_n , t_n , and F_n are taken to be normalized, that is, the unit in which they are measured is chosen such that the generation rate of the messages is 1.

We find the optimum $F(\cdot, \cdot)$ by the successive approximation method of solving undiscounted infinite horizon Markovian decision theory problems [15]. That is, we assume that the process will end after N more transmissions, and define a reward equal to 1 for a successful transmission, and equal to 0 otherwise. We then assign to each state (s, t) at time N a value $V_{(s,t)}(N)$, which is the maximum expected reward in N transmissions when starting in state (s, t) . From dynamic programming principles this is equal to the sum of the maximum over all F of the conditional expected reward on the next state transition and the conditional expected value of $V_{(s',t')}(N-1)$, where the expected values are conditioned on the state (s, t) and (s', t') is the subsequent state. (Since this definition

is recursive, it is necessary to define $V_{(s,t)}(0) = 0$.) Hence,

$$\begin{aligned} & V_{(s,t)}(N+1) \\ &= \max_{F_N(s,t) \geq 0} [\Pr(\theta_N = 0 | s, t, F_N(s, t)) \\ & \quad \cdot V_{(s-F_N(s,t), t-F_N(s,t))}(N) \\ & \quad + \Pr(\theta_N = 1 | s, t, F_N(s, t)) \\ & \quad \cdot (1 + V_{(t-F_N(s,t), \infty)}(N)) + \Pr(\theta_N = 2 | s, t, F_N(s, t)) \\ & \quad \cdot V_{(\min(s, F_N(s,t)), F_N(s,t))}(N)] \end{aligned}$$

with $V_{(s,t)}(0) = 0$ for all s, t . As N goes to infinity, the differences $V_{(s,t)}(N+1) - V_{(s,t)}(N)$ converge to the maximum throughput λ^* and the sequence of functions $F_N(s, t)$ converge to the function $F(s, t)$ that achieves the throughput λ^* .

The value functions $V_{(s,t)}(N)$ and the control functions $F_N(s, t)$ were evaluated numerically for a finite number of points over an appropriately bounded, discretized state space. Details of this work may be found in [16]. Several interesting conclusions were reached.

First, the optimal $F(s, t)$ is never greater than s , except $F(0, \infty) \cong 0.057$, so that all states (s, t) with $s \neq t$ or $t \neq \infty$ are transient.¹ In addition, although a threshold s_T exists such that $s > s_T$ implies $F(s, t) < s$, if the optimal F is used for all transmissions, we can never enter a state where s exceeds this threshold. Hence, for all nontransient states, we have $F(s, t) = s$.

The optimal $F(\infty, \infty)$ is 1.275, so that all states with $\infty > t > s > 1.275$ are transient. All that is required now to complete the specifications of $F(\cdot, \cdot)$ are the values of $F(s, s)$ for $0 < s < 1.275$. These are given in Table I. Observe that $F(s, s)$ is approximately $s/2$. Hence, this algorithm is very close to Gallager's binary splitting algorithm described in Section II. Indeed, the improvement in the throughput of this algorithm over the other is negligible: 0.48776 versus 0.48711. The binary splitting algorithm would be optimal if, whenever a collision occurred, the collision were known to involve exactly two messages. But because there is some positive probability of more than two messages colliding, the optimal $F(s, s)$ is slightly less than $s/2$.

We note, however, that the first remark above (i.e., that the optimal F satisfies $F(s, t) \leq s$) does not hold for finite horizon ($N < \infty$) problems. For these, the optimal $F(s, \infty)$ may be larger than s for small N . The optimal $F_N(s, \infty)$ is shown in Fig. 2 for $N = 3, 4$, and 5. We note that for each N , there is a large discontinuity in F , i.e., a threshold $s_T(N)$ such that, for $s < s_T(N)$, $F_N(s, \infty) > s$ and such that for $s > s_T(N)$, $F_N(s, \infty) = s$. The threshold decreases with increasing N , becoming smaller than the grid size used (0.01) of the discretized state space for $N > 5$. No similar behavior was observed for $F(s, t)$, $t < \infty$, probably because the numerical optimization did not consider (transient) states in the region where the phenomenon would occur.

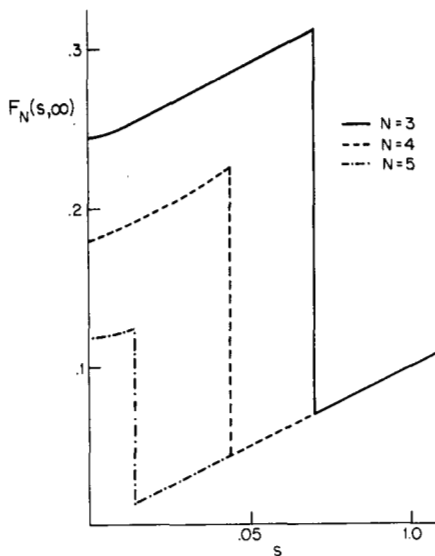
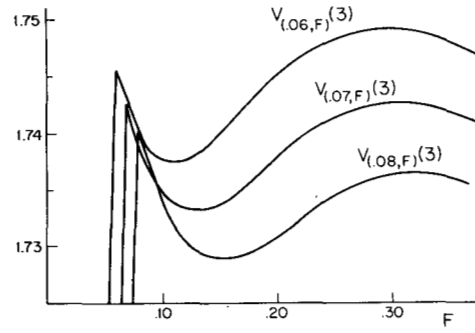
The existence of this discontinuity in F is surprising and the reason for it is worth discussing. In Fig. 3, we see the value functions at $N = 3$ plotted as a function of F for three states in the neighborhood of the threshold. Each of these functions have two local maxima, one at $F = s$ and one for $F > s$. For the state $(0.06, \infty)$, the maximum occurs at $F = 0.3$. For $(0.07, \infty)$, the two maxima are equal. For $(0.08, \infty)$, the maximum is at $F = 0.08$. Hence, we have a threshold at about $s = 0.07$.

As mentioned in the Introduction, a similar algorithm has been presented independently by Tsybakov and Mikhailov [9]. Their version is somewhat more restrictive than ours,

¹ $F(s, \infty) \cong 0.05$ for s in $[0, \epsilon]$, with ϵ about $2 \cdot 10^{-3}$ (smaller than our grid size), would increase λ^* by about $3 \cdot 10^{-7}$. See [27].

TABLE I

s	F(s,s)	s	F(s,s)	s	F(s,s)
0.01	0.005	0.44	0.214	0.87	0.410
0.02	0.010	0.45	0.218	0.88	0.415
0.03	0.015	0.46	0.223	0.89	0.419
0.04	0.020	0.47	0.228	0.90	0.423
0.05	0.025	0.48	0.232	0.91	0.428
0.06	0.030	0.49	0.237	0.92	0.432
0.07	0.035	0.50	0.242	0.93	0.437
0.08	0.040	0.51	0.246	0.94	0.441
0.09	0.045	0.52	0.251	0.95	0.445
0.10	0.050	0.53	0.256	0.96	0.450
0.11	0.055	0.54	0.260	0.97	0.454
0.12	0.060	0.55	0.265	0.98	0.458
0.13	0.064	0.56	0.270	0.99	0.463
0.14	0.069	0.57	0.274	1.00	0.467
0.15	0.074	0.58	0.279	1.01	0.471
0.16	0.079	0.59	0.284	1.02	0.476
0.17	0.084	0.60	0.288	1.03	0.480
0.18	0.089	0.61	0.293	1.04	0.484
0.19	0.094	0.62	0.297	1.05	0.489
0.20	0.099	0.63	0.302	1.06	0.493
0.21	0.104	0.64	0.307	1.07	0.497
0.22	0.108	0.65	0.311	1.08	0.501
0.23	0.113	0.66	0.316	1.09	0.506
0.24	0.118	0.67	0.320	1.10	0.510
0.25	0.123	0.68	0.325	1.11	0.514
0.26	0.128	0.69	0.329	1.12	0.519
0.27	0.133	0.70	0.334	1.13	0.523
0.28	0.137	0.71	0.338	1.14	0.527
0.29	0.142	0.72	0.343	1.15	0.531
0.30	0.147	0.73	0.347	1.16	0.536
0.31	0.152	0.74	0.352	1.17	0.540
0.32	0.157	0.75	0.357	1.18	0.544
0.33	0.161	0.76	0.361	1.19	0.548
0.34	0.166	0.77	0.365	1.20	0.552
0.35	0.171	0.78	0.370	1.21	0.557
0.36	0.176	0.79	0.374	1.22	0.561
0.37	0.181	0.80	0.379	1.23	0.565
0.38	0.185	0.81	0.383	1.24	0.569
0.39	0.190	0.82	0.388	1.25	0.573
0.40	0.195	0.83	0.392	1.26	0.578
0.41	0.199	0.84	0.397	1.27	0.582
0.42	0.204	0.85	0.401	1.28	0.586
0.43	0.209	0.86	0.406	1.29	0.590

Fig. 2. $F_N(s, \infty)$ as a function s for $N = 3, 4, 5$.Fig. 3. $V_{(s,F)}(3)$ as a function of F .

as they impose the condition that $F(s, \infty) = s$. That is, when an interval is known to contain at least one generation time, that interval is chosen as the next enabled interval. Hence, the only recurrent states have the form (s, s) , (s, ∞) , and (∞, ∞) , exactly as in Gallager's binary splitting algorithm. Thus, only $F(\infty, \infty) = \tau$ and $F(s, s)$ for $s \leq \infty$ need to be determined. They did not actually find the optimal F for this subclass of algorithms, but stated that if $F(\infty, \infty) = 1.275$, $F(1.275, 1.275) = 0.584$ and $F(s, s) = 0.485s$ for all $s \leq 0.584$, then the throughput is 0.48778. Using the same values for $F(\cdot, \cdot)$, we calculate a throughput of 0.48773. This discrepancy is unexplained but, since the results differ only in the fifth decimal place, is not very important.

IV. REAL-TIME IMPLEMENTATION

In the idealized version of Section III it was assumed that all messages were generated before the algorithm started. In practice, generations and transmissions would take place concurrently. Hence, the original algorithm is not causal, in the sense that it sometimes specifies that messages should be transmitted before having been generated. This can be remedied by defining

$$F_n(s_n, t_n, y_n) = \min [F(s_n, t_n), n - y_n]. \quad (2)$$

The quantity $n - y_n$ that appears above, we call the *lag* of the algorithm. That is, the lag is length of time during which messages have already been generated but not yet successfully transmitted.

To analyze real-time performance parameters, like message delay, one must study the Markov process (s_n, t_n, y_n) , as the process (s_n, t_n) is no longer Markovian. This appears to be extremely complicated when the boundary condition (2) is imposed. However, some simple statements can be made regarding the behavior of the lag $n - y_n$ as a function of λ (in this section s, t , and y are expressed in terms of "slot" units; λ is in units of messages per slot).

Let $k(m)$ denote the time when $(s, t) = (\infty, \infty)$ for the m th time (that is, $k(m) = \min \{k | k > k(m-1), (s_k, t_k) = (\infty, \infty)\}$). Note that if the probability of success at a step has a positive lower bound, then $\Pr(k(m) - k(m-1) > x)$ decreases at least geometrically with x , and $E[k(m) - k(m-1)]$ is finite. Moreover, the "drifts" $y_k(m) - y_k(m-1)$ are independent. By a renewal argument, we can show

$$E(y_{k(m+1)} - y_{k(m)}) = \frac{\lambda^*}{\lambda} E(k(m+1) - k(m))$$

where λ^* is the throughput of the algorithm, and is a function of λ . This follows since the throughput is the expected number of message generations in $y_{k(m+1)} - y_{k(m)}$ divided by the expected number of trials $E(k(m+1) - k(m))$.

The expected difference between the lag at times $y_{k(m+1)}$

and $y_{k(m)}$ is

$$\begin{aligned} E[k(m+1) - k(m)] &= E[y_{k(m+1)} - y_{k(m)}] \\ &= (1 - \lambda^*/\lambda)E[k(m+1) - k(m)]. \end{aligned}$$

Hence, in the idealized version, as long as $\lambda < 0.48776$, the expected changes in lag are negative, and the algorithm will repeatedly select enabled intervals $[y_n, y_n + F_n]$ where $y_n + F_n > n$.

When (2) holds, it is easy to see that $\lambda^* = \min(\lambda, 0.48776)$. Clearly $\lambda^* \leq \lambda$. If $\lambda^* < \lambda$, then the expected change in lag from $y_{k(m)}$ to $y_{k(m+1)}$ is positive and the lag increases without bound as n goes to infinity. But whenever the lag is greater than or equal to $\tau = F(\infty, \infty)$, the choice of F_n is the same as for the unconstrained algorithm, and the throughput is 0.48776. Hence, when $\lambda > 0.48776$, $\lambda^* = 0.48776$ and the lag goes to infinity.

When $\lambda \leq 0.48776$, $\lambda^* = \lambda$, and the expected change in lag is 0. Furthermore, from the observations made above that $k(m) - k(m-1)$ has a geometric tail distribution and the expected change in lag is negative when the lag exceeds τ , we can show, using the results in [17], that the probability that the lag is greater than x has an upper bound that decreases exponentially with x .

It is reassuring to note that, even when the generation rate of the messages exceeds the throughput of the algorithm, it will continue to transmit successfully at its maximum throughput.

V. UNEQUAL OBSERVATION TIMES

In fact, many multiaccess communication systems differ from the model introduced in Section I in that the times necessary to learn the transmission outcomes depend on the outcomes. We denote by t_0 , t_1 , and t_2 , respectively, the times necessary to learn that the channel was idle, or that a success or a collision occurred.

For example, carrier sense radio systems [2] can detect idles quickly (no carrier present), while they rely on error-detecting codes and the transmissions of acknowledgments to distinguish between successes and collisions; thus, $t_0 \ll t_1 = t_2$. In addition, some cable broadcast systems [3] have a listen-while-transmit feature that allows the quick abortion of transmissions resulting in collisions; thus, $t_0 \cong t_2 \ll t_1$. Reservation systems also fall in the last category.

The general algorithm outlined in Section II and the remarks about its Markovian nature remain valid for this modified problem, but the reward function and the maximization in Section III are not appropriate. A better goal is to minimize the expected time to send a message, i.e.,

$$\begin{aligned} & \lim_{N \rightarrow \infty} \frac{E\left(\sum_{i=1}^N \sum_{j=0}^2 t_j I(\theta_i = j)\right)}{E\left(\sum_{i=1}^N I(\theta_i = 1)\right)} \\ &= t_1 + \sqrt{t_0 t_2} \lim_{N \rightarrow \infty} \frac{E\left(\sum_{i=1}^N \sqrt{\frac{t_0}{t_2}} I(\theta_i = 0) + \sqrt{\frac{t_2}{t_0}} I(\theta_i = 2)\right)}{E\left(\sum_{i=1}^N I(\theta_i = 1)\right)}. \end{aligned}$$

The second term on the right-hand side can be interpreted as the expected *time overhead* per message. Its second factor depends only on t_0/t_2 for a given $F(\cdot, \cdot)$. It will be denoted by c and should be minimized over $F(\cdot, \cdot)$ for a given t_0/t_2 .

The throughput (i.e., the fraction of time that successful transmissions are in progress) can be obtained from c by the relation

$$\text{throughput} = t_1 / (t_1 + \sqrt{t_0 t_2} c).$$

The optimization of the general algorithm under this formulation for a large number of values of t_0/t_2 is time consuming. It is greatly simplified if we consider only those $F(\cdot, \cdot)$ such that $F(s, t) \leq s$. The only recurrent states are then of the form (s, s) , or (s, ∞) ; see above. Note that the optimal F found in Section III belonged to the restricted class, but we do not claim that this will hold true for all t_0/t_2 . We will now show how to proceed with the optimization.

By a renewal argument,

$$c = \frac{E\left(\sum_{i=1}^b \sqrt{\frac{t_0}{t_2}} I(\theta_i = 0) + \sqrt{\frac{t_2}{t_0}} I(\theta_i = 2)\right)}{E\left(\sum_{i=1}^b I(\theta_i = 1)\right)}$$

where in the right-hand side one assumes that $(s_1, t_1) = (\infty, \infty)$ and that b is the time of first return to (∞, ∞) .

Let us now guess that $\hat{c}$ is the minimum of c over all restricted $F(\cdot, \cdot)$, and consider the function

$$\begin{aligned} V(s, t) &= E\left[\sum_{i=1}^b \sqrt{\frac{t_0}{t_2}} I(\theta_i = 0) + \sqrt{\frac{t_2}{t_0}} I(\theta_i = 2)\right. \\ &\quad \left. - \hat{c} I(\theta_i = 1) \mid (s_1, t_1) = (s, t)\right]. \end{aligned}$$

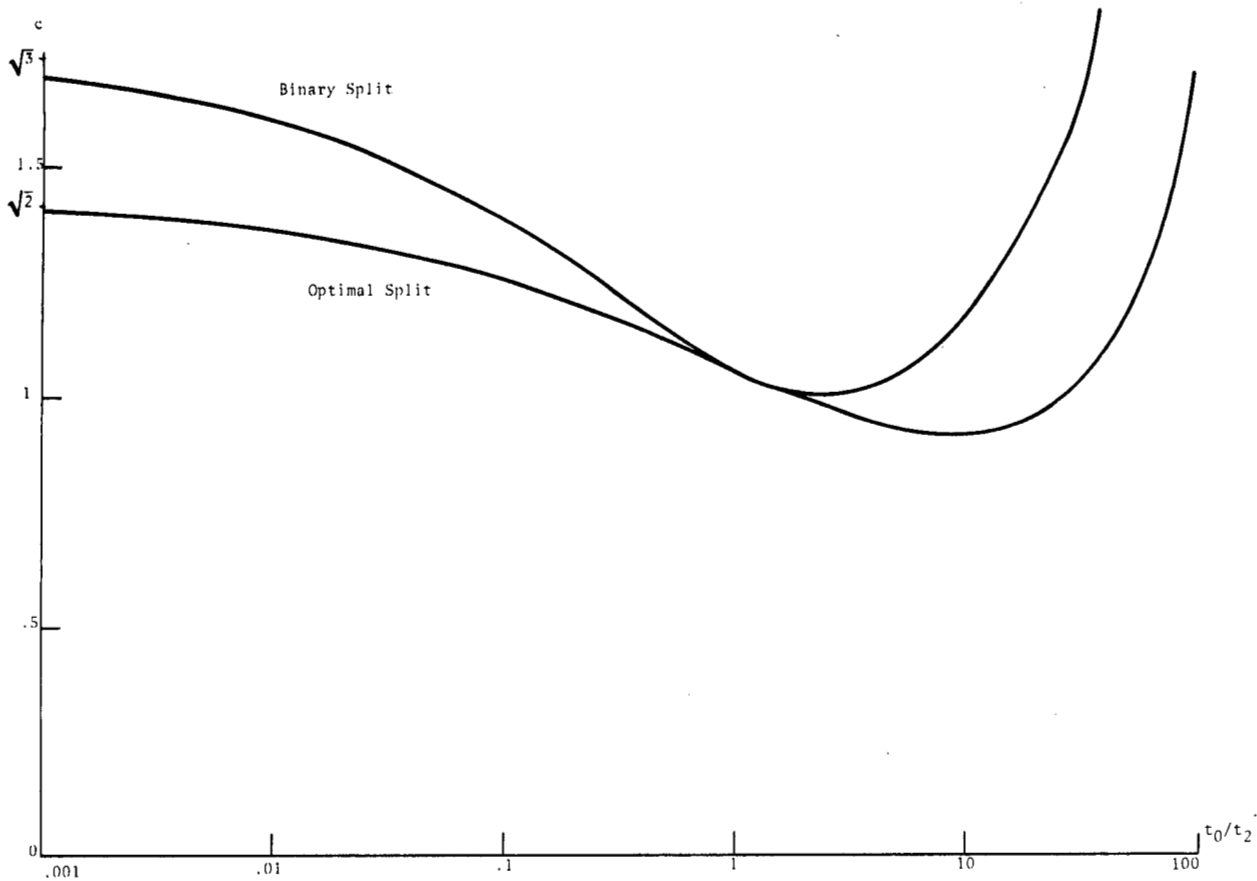
Because s_{n+1} is either equal to $F(\infty, \infty)$ or is less than or equal to s_n , $V(s, s)$ and $V(s, \infty)$ can be written as linear combinations of $V(s', s')$, $V(s', \infty)$, and $\hat{c}$, where $s' \leq \min(s, F(\infty, \infty))$. The coefficients of the linear combination are the conditional probabilities of the outcomes, $\theta = 0, 1$, or 2 , and are functions on (s, t) and $F(s, t)$. It is straightforward [13] to minimize $V(s, s)$ and $V(s, \infty)$ recursively for increasing s , and to obtain the minimum value of $V(\infty, \infty)$.

If the minimum value is 0, $\hat{c}$ was guessed correctly and is the minimum value of c . If the minimum value of $V(\infty, \infty)$ is positive (negative), $\hat{c}$ was guessed too small (large), and the minimization of $V(\cdot)$ must be repeated with a new $\hat{c}$. The resulting minimum value of c is shown in Fig. 4, as a function of t_0/t_2 . Note that the behavior of c changes for large t_0/t_2 , as "idles" cannot be avoided as easily as "collisions."

The optimal value of $F(\infty, \infty)$ is always in the vicinity of $1.3 \sqrt{t_0/t_2}$, except for large t_0/t_2 . For very small t_0/t_2 , the constant 1.3 becomes precisely $\sqrt{2}$ and $\sqrt{4/3}$ for binary and optimal splitting, respectively.

VI. THE GENERAL FGFR ALGORITHM

We note that the previously analyzed algorithm is a first-generated first-transmitted algorithm, which is a desirable fairness property. However, fairness only requires that algorithms have the first-generated first-received-correctly (FGFR) property, and our algorithms are not the most general in that respect. The results of Section III suggest some conjectures with regard to the most general FGFR algorithm, which we describe in this section.

Fig. 4. c as a function of t_0/t_2 .

For an algorithm to be FGFR, it must satisfy one of two conditions. Either it does not allow a message to be transmitted when other messages with earlier generation times must wait, or, if it does, the probability of successful transmission must be zero.

Suppose that we are using an FGFR algorithm to resolve conflicts and that all messages with generation times prior to y_n have been successfully transmitted. We will call the set of messages generated after y_n the "queue." The algorithm selects some subset of the queue to transmit in the next slot, which we call the enabled set. Then the most general form of an enabled set which satisfies the first condition above will clearly be the set of all messages generated in an interval of the form $[y_n, y_n + F_n]$.

Now let us consider enabled sets satisfying the second condition. Suppose we have a subset of the queue, S_1 , which is known to contain at least one message, but it is not known whether it contains more. Consider the enabled set which is the union of S_1 and some subset of the queue, S_2 . If the subset of S_2 which is disjoint from S_1 is not empty, then a conflict occurs and the second condition holds. If the subset of S_2 which is disjoint from S_1 is empty, then S_1 must be the set of messages generated in an interval of the form $[y_n, y_n + F_n]$ for the first condition to hold. Assuming, without loss of optimality, that the algorithm never chooses an enabled set that is known to contain a conflict, we cannot know in advance if the subset of S_2 disjoint from S_1 is nonempty. Hence, S_1 must be the set of messages generated in an interval of the form $[y_n, y_n + F_n]$ to ensure that one of the two conditions holds.

The general FGFR algorithm must use enabled sets satisfying one of the two conditions described above. It differs from the basic algorithm of Section II only by permitting the use of enabled sets satisfying the second condition. But when we consider that for our algorithm the optimal $F_n(s, \infty)$ is

s [that is, when an interval $[y_n, y_n + F_n]$ is known to contain at least one generation time, the optimal enabled interval is just $[y_n, y_n + F_n]$], it seems unlikely that using a more general enabled set of the form $[y_n, y_n + F_n] \cup S_2$ would offer any improvement. We cannot at present prove this conjecture, however, since allowing this type of enabled set makes it no longer possible to characterize the algorithm as a Markov process with just two (or even a finite number of) state variables. Furthermore, since Cruz [18] has shown that the maximum throughput for any FGFR algorithm must be less than 0.5, any improvement in throughput would be too small to justify the additional complexity of implementing such an algorithm.

VII. NOISY FEEDBACK

The previous algorithm assumed that the transmission outcomes were perfectly observed by all sources. This assumption is critical. One verifies easily that if an idle at time n is falsely observed as a collision, then the algorithm will deadlock. The algorithm will behave as if there is a conflict in the interval $[y_n, y_n + t_n]$ and when the next enabled interval $[y_n, y_n + F_n]$ produces an idle, the algorithm will proceed as if there was a conflict in the interval $[y_n + F_n, y_n + t_n]$. Hence, $y_n + t_n$ will remain constant while t_n goes to zero.

D. Ryter [19] has examined the problem of noisy feedback, where the noise can cause idles or successes to be observed as collisions. He showed that the binary splitting algorithm in Section II can be modified to work properly. The essential modification is the introduction of a threshold value. If t_n is smaller than the threshold, then the algorithm becomes nonstationary, in the sense that it alternates between using $F(s, s) = s$ and $F(s, s) = s/2$, first seeking confirmation that a collision really occurred, then trying to resolve it. The analysis and optimization are too long to be reported here. The main result is that with the proper choice of parameters,

the throughput behaves roughly like $0.487 - p$, where p is the probability of false collision indication. Massey reported a similar analysis in [20].

VIII. FINAL COMMENTS

The main results of this paper are the description and analysis of an access algorithm for the channel model described in Section I, with infinitely many sources. Its throughput is 0.48776, the largest known for any algorithm to this day. Much research has been done to determine the upper bounds on the possible throughput [21]–[25]. Tsybakov and Mikhailov [26] have recently shown that no algorithm can have a throughput greater than 0.5874, and it is widely believed that the best achievable throughput is in the neighborhood of 0.5. However, throughputs arbitrarily close to 1 are possible, at the expense of high average message delay, when the number of sources is finite.

We have also shown how the algorithm can be modified in the cases of variable transmission times and noisy feedback. Upper bounds on the throughput for the case of variable transmission times are given by Humblet in [23].

Finally, it should be pointed out that although the algorithm presented here uses the message generation times to specify when they should be transmitted, this is not necessary. Another algorithm can be described, with the same throughput and expected time overhead per message, where sources generate random numbers to determine if they should transmit. Of course, real-time properties, like first-generated first-transmitted, will not be conserved.

ACKNOWLEDGMENT

We would like to thank the anonymous reviewers for their many helpful comments and suggestions, and the painstaking care with which they studied and edited the manuscript. We also thank J. Hui for performing the numerical optimization that led to Fig. 4.

REFERENCES

- [1] L. G. Roberts, "Aloha packet system with and without slots and capture," ASS, Note 8, June 26, 1972.
- [2] L. Kleinrock and F. Tobagi, "Carrier sense multiple access for packet switched radio channels," in *Proc. Int. Conf. Commun.*, Minneapolis, MN, June 1974, pp. 21B-1-21B-7.
- [3] R. M. Metcalfe and D. R. Boggs, "Ethernet: Distributed packet switching for local computer networks," *Commun. Ass. Comput. Mach.*, vol. 19, pp. 396-404, July 1976.
- [4] L. Kleinrock and Y. Yemini, "An optimal adaptive scheme for multiple access broadcast communication," in *Proc. Int. Conf. Commun.*, 1978, pp. 7.2.1-7.2.5.
- [5] L. Kleinrock and S. S. Lam, "Packet switching in a multiaccess broadcast channel: Performance evaluation," *IEEE Trans. Commun.*, vol. COM-23, pp. 410-423, Apr. 1975.
- [6] F. Schoute, "Decentralized control in packet switched satellite communication," *IEEE Trans. Automat. Contr.*, vol. AC-23, pp. 362-371, Apr. 1978.
- [7] P. Varaiya and J. Walrand, "Decentralized control in packet switched satellite communication," *IEEE Trans. Automat. Contr.*, vol. AC-24, pp. 794-796, Oct. 1979.
- [8] —, "A minimum principle for dynamic teams," presented at 3rd Kingston Conf. Differential Games, Contr. Theory, Univ. Rhode Island, Kingston, June 1978.
- [9] B. S. Tsybakov and B. A. Mikhailov, "Random multiple packet access: Part-and-try algorithm," *Probl. Peredach. Inform.*, vol. 16, pp. 65-79, Oct.-Dec. 1980.
- [10] R. G. Gallager, "Conflict resolution in random access broadcast networks," in *Proc. AFOSR Workshop Commun. Theory Appl.*, Provincetown, MA, Sept. 17-20, 1978, pp. 74-76.
- [11] J. F. Hayes, "An adaptive technique for local distribution," *IEEE Trans. Commun.*, vol. COM-26, pp. 1178-1186, Aug. 1978.
- [12] J. Capetanakis, "The multiple access broadcast channel: Protocol and capacity considerations," Ph.D. dissertation, Massachusetts Inst. Technol., Cambridge, 1977.
- [13] D. F. Towsley and J. K. Wolf, "An application of group testing to the design of multi-user access protocols," Dep. Elec. Eng. Comput. Eng., Univ. Massachusetts, Amherst, Tech. Rep., 1981.
- [14] J. Mosely and P. A. Humblet, "A class of efficient contention resolution algorithms for multiple access channels," Lab. Inform. Decision Syst., M.I.T., Cambridge, MA, Rep. LIDS-P-1194, Dec. 1982.
- [15] R. Howard, *Dynamic Programming and Markov Processes*. Cambridge, MA: M.I.T. Press, 1960.
- [16] J. Mosely, "An efficient contention resolution algorithm for multiple access channels," Lab. Inform. Decision Syst., M.I.T., Cambridge, MA, Rep. LIDS-TH-918, May 1979.
- [17] B. Hajek, "Hitting time and occupation time bounds implied by drift analysis with applications," Co-ord. Sci. Lab., Dep. Elec. Eng., Univ. Illinois, Urbana, Tech. Rep., 1981.
- [18] R. L. Cruz, "Protocols for multi-access channels with continuous entry and noisy feedback," Lab. Inform. Decision Syst., M.I.T., Cambridge, MA, Rep. LIDS-TH-1213, June 1982.
- [19] D. M. Rytter, "A conflict resolution algorithm for noisy multiaccess channels," Lab. Inform. Decision Syst., M.I.T., Cambridge, MA, Rep. LIDS-TH-1007, June 1980.
- [20] J. L. Massey, "Collision resolution algorithms and random access communications," in *Multi-user Communications* (CISM Courses and Lectures No. 265). New York: Springer-Verlag, 1981, pp. 73-137.
- [21] N. Pippenger, "Bounds on the performance of protocols for a multiple access broadcast channel," *IEEE Trans. Inform. Theory*, vol. IT-27, pp. 145-151, Mar. 1981.
- [22] P. A. Humblet, "Bounds on the utilization of ALOHA-like multiple access broadcast channels," Lab. Inform. Decision Syst., M.I.T., Cambridge, MA, Rep. LIDS-P-1000, June 1980.
- [23] —, "Data communication and information theory," in *Proc. Nat. Telecommun. Conf.*, Houston, TX, Dec. 1980, pp. 20.3.1-20.3.5.
- [24] M. L. Molle, "On the capacity of infinite population multiple access protocol," *IEEE Trans. Inform. Theory*, vol. IT-28, pp. 396-401, May 1982.
- [25] R. Cruz and B. Hajek, "A new upper bound to the throughput of a multi-access broadcast channel," *IEEE Trans. Inform. Theory*, vol. IT-28, pp. 402-405, May 1982.
- [26] B. S. Tsybakov and V. A. Mikhailov, "An upper bound to capacity of random multiple access systems," presented at Inform Theory Symp., Santa Monica, CA, 1981; also in *Probl. Peredach. Inform.*, vol. 17, pp. 90-95, Jan.-Mar. 1981.
- [27] N. S. Vvedenskaya and M. S. Pinsker, "Nonoptimality of the part-and-try algorithm," in *Abstr. Papers, Int. Workshop "Convolutional Codes; Multi-User Commun."*, Sochi, U.S.S.R., May 30-June 6, 1983, pp. 141-144.



Jeannine Mosely (S'79-M'82) received the B.A. degree in mathematics and the B.S. degree in electrical engineering from the University of Illinois, Urbana, in 1974 and 1977, respectively, and the S.M. and Ph.D. degrees in electrical engineering from the Massachusetts Institute of Technology, Cambridge, in 1979 and 1984, respectively.

She has worked during the summer at M.I.T. Lincoln Laboratories, Lexington, MA, from 1978 to 1980, and has consulted for various software companies, helping to develop educational material.

Her professional interests include computer communication networks and probabilistic systems. Her other interests include polyhedra, and she is currently working on a book of recreational mathematics on that subject.



Pierre A. Humblet (S'72-M'77) was born in Brussels, Belgium on July 13, 1949. He received the Ingénieur Electricien degree from the Université Catholique de Louvain, Louvain, Belgium, in 1973, and the M.S., E.E., and Ph.D. degrees in electrical engineering and computer science from the Massachusetts Institute of Technology, Cambridge, in 1975, 1975, and 1978, respectively.

He has remained at M.I.T., where he is now an Associate Professor of Electrical Engineering. He holds the Nippon Electric Corporation Professor-

ship of Computers and Communications. His current research interests include distributed systems, computer communications networks, and multiaccess systems.